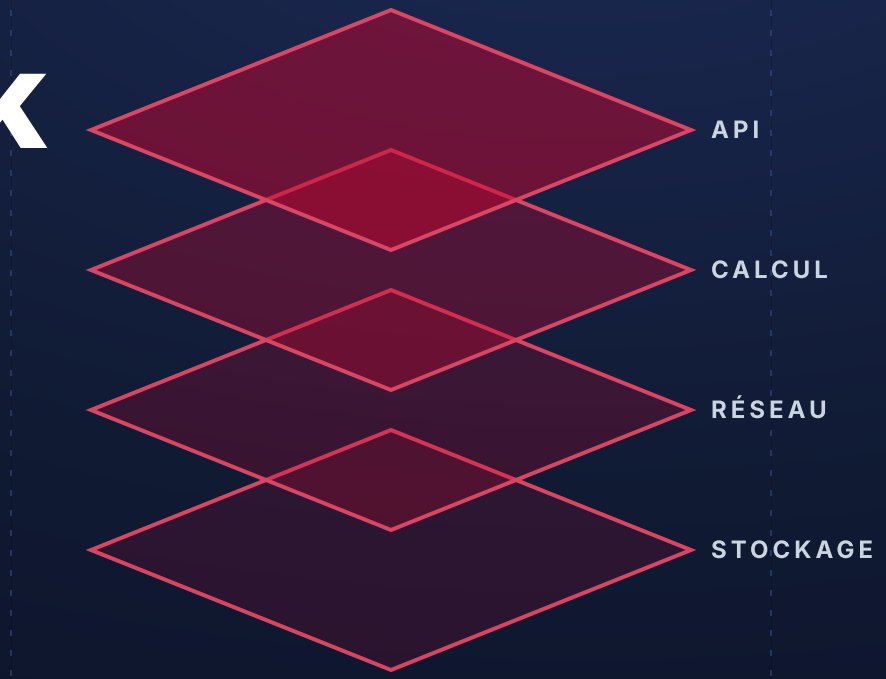
COMPRENDRE OPENSTACK · MODULE 1

Panorama d'OpenStack

Le cloud, le projet, les alternatives, l'architecture d'ensemble

- Cloud, IaaS et PaaS
- VMware · OpenShift · clouds publics
- Le projet OpenStack
- Composants et interactions

Version de référence : OpenStack 2026.2 « Hibiscus » (30 septembre 2026)



INTRODUCTION

Objectif et déroulé

Objectif : comprendre comment les composants d'OpenStack fonctionnent et interagissent entre eux, et savoir situer OpenStack face aux autres solutions d'infrastructure.

À l'issue de ce module, vous saurez :

- Définir le cloud computing et distinguer **IaaS, PaaS, CaaS et SaaS** par le partage des responsabilités.
- Expliquer ce qu'est OpenStack, d'où il vient, qui le gouverne et comment il est publié.
- **Comparer** OpenStack à VMware, à OpenShift et aux clouds publics, service par service.
- Décrire l'**architecture** : plan de contrôle, nœuds, API REST, bus de messages, bases de données.
- Présenter le rôle de chaque **service** (Keystone, Nova, Neutron, Cinder, Glance, Swift, Heat…).
- Dérouler pas à pas la **création d'une instance** à travers tous les services.

1 · Positionnement du cloud

Définition, modèles de service et de déploiement, virtualisation ou cloud, raisons d'un cloud privé.

2 · Le projet OpenStack

Définition, histoire, gouvernance, cycle de publication, utilisateurs, distributions.

3 · OpenStack face aux alternatives

VMware, OpenShift et Kubernetes, clouds publics, grille de choix.

4 · Architecture et composants

Vue d'ensemble, socle technique, services un par un, flux complet, haute disponibilité.

01

Positionnement du cloud

Avant de parler d'OpenStack, il faut un vocabulaire commun : ce qu'est réellement un cloud, ce qui le distingue de la virtualisation, et qui est responsable de quoi.

- Les cinq caractéristiques d'un cloud (NIST)
- IaaS, PaaS, SaaS : partage des responsabilités
- Cloud public, privé, hybride
- Du serveur physique au cloud
- CaaS, FaaS : le continuum des services
- Virtualisation ou cloud ? « Pets » et « cattle »

Qu'est-ce qu'un cloud ? Les cinq caractéristiques du NIST

La définition de référence est celle du **NIST (SP 800-145)** : un modèle d'accès par le réseau, à la demande, à un pool partagé de ressources configurables, provisionnées et libérées rapidement avec un minimum d'intervention du fournisseur.

Libre-service à la demande

L'utilisateur provisionne seul, sans ticket ni intervention humaine.

OpenStack : API REST, CLI, Horizon

Accès réseau étendu

Les services sont accessibles par des protocoles standards, depuis n'importe quel client.

OpenStack : endpoints HTTPS publiés dans le catalogue

Mise en commun des ressources

Un même pool sert plusieurs clients isolés logiquement (multi-tenant).

OpenStack : domaines, projets, réseaux isolés

Élasticité rapide

Les ressources s'ajoutent et se retirent en quelques secondes, parfois automatiquement.

OpenStack : création par API, autoscaling Heat

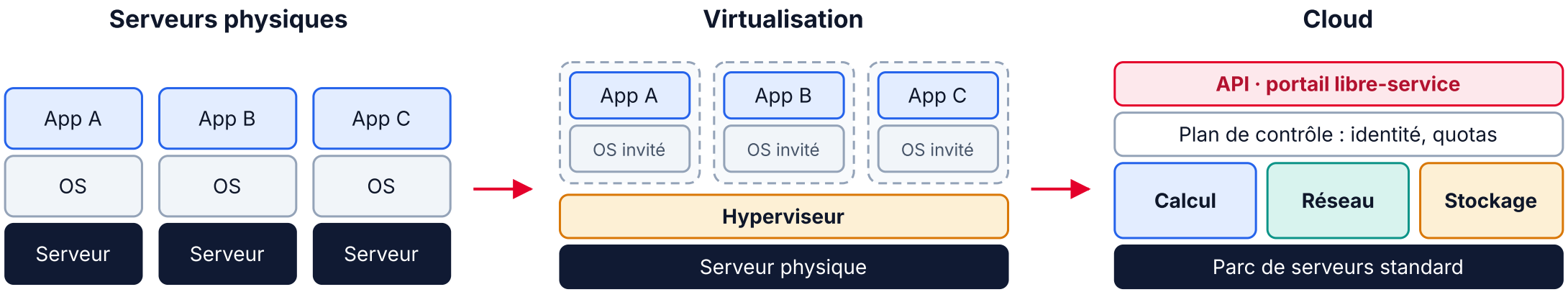
Service mesuré

L'usage est compté, plafonné et restituable (facturation ou refacturation).

OpenStack : quotas, Ceilometer, CloudKitty

> Si l'une de ces cinq propriétés manque — typiquement le libre-service ou la mesure — on parle d'**hébergement** ou de **virtualisation**, pas de cloud. C'est le critère à garder en tête pour toutes les comparaisons qui suivent.

Du serveur physique au cloud : trois générations d'infrastructure



- Une application = un serveur dédié
- Taux d'utilisation faible (souvent < 20 %)
- Achat, câblage, installation à chaque besoin
- Panne matérielle = panne applicative

Délai de mise à disposition
plusieurs semaines

- Consolidation : N machines virtuelles par hôte
- Pilotage par l'équipe infrastructure
- Demande par ticket, création manuelle
- Disponibilité assurée par l'infrastructure

Délai de mise à disposition
quelques heures à quelques jours

- Libre-service par API, multi-tenant
- Pools abstraits, quotas, usage mesuré
- Infrastructure décrite par du code
- Résilience portée par l'application

Délai de mise à disposition
quelques secondes à quelques minutes

IaaS, PaaS, SaaS : qui gère quoi ?

	Sur site infrastructure traditionnelle	IaaS OpenStack, AWS EC2, Azure VM	PaaS OpenShift, Azure App Service	SaaS Microsoft 365, Salesforce
Applications	vous	vous	vous	fournisseur
Données	vous	vous	vous	fournisseur (hébergement)
Environnement d'exécution	vous	vous	fournisseur	fournisseur
Middleware	vous	vous	fournisseur	fournisseur
Système d'exploitation	vous	vous	fournisseur	fournisseur
Virtualisation	vous	fournisseur	fournisseur	fournisseur
Serveurs	vous	fournisseur	fournisseur	fournisseur
Stockage	vous	fournisseur	fournisseur	fournisseur
Réseau physique	vous	fournisseur	fournisseur	fournisseur

Point clé pour OpenStack : dans un cloud privé, votre organisation tient les deux rôles. L'**équipe plateforme** est le « fournisseur » (elle exploite OpenStack et tout ce qui est en dessous) ; les **équipes projets** sont les « clients » qui consomment de l'IaaS en libre-service.

L'IaaS en détail : l'infrastructure devient une API

Infrastructure as a Service : le fournisseur expose des ressources d'infrastructure **virtualisées et programmables**. Le client assemble lui-même ses machines, ses réseaux et son stockage, puis installe et exploite tout ce qui tourne dessus.

- **Ce que vous obtenez :** machines virtuelles, réseaux privés, routeurs, adresses IP publiques, volumes, stockage objet, images système.
- **Ce qui reste à votre charge :** système d'exploitation, correctifs, middleware, sauvegardes applicatives, sécurité de l'instance.
- **Unité de consommation :** vCPU, Go de RAM, Go de disque, adresses IP, trafic — par heure ou à la seconde.
- **Atout :** contrôle maximal, compatible avec presque toutes les applications existantes.
- **Contrepartie :** charge d'exploitation la plus élevée des modèles cloud.

Brique IaaS	Service OpenStack	Ressources exposées
Calcul	**Nova**	Instances, gabarits (flavors), paires de clés
Réseau	**Neutron**	Réseaux, sous-réseaux, routeurs, IP flottantes, groupes de sécurité
Stockage bloc	**Cinder**	Volumes, instantanés, sauvegardes
Stockage objet	**Swift**	Comptes, conteneurs, objets
Images	**Glance**	Images système, instantanés d'instances
Identité	**Keystone**	Utilisateurs, projets, rôles, catalogue

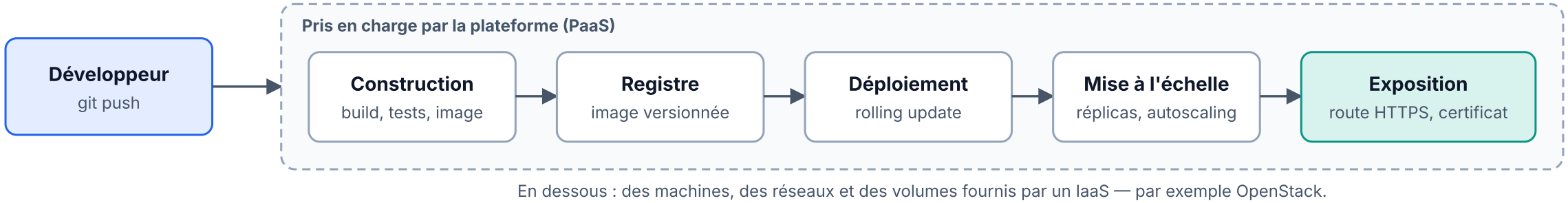
> Chaque brique IaaS d'OpenStack a son équivalent chez les clouds publics (EC2, VPC, EBS, S3…) : la correspondance complète est donnée en partie 3.

Le PaaS en détail : la plateforme gère l'exécution

Platform as a Service : le développeur fournit son **code** (ou une image de conteneur) ; la plateforme prend en charge la construction, le déploiement, le routage, la mise à l'échelle et la supervision.

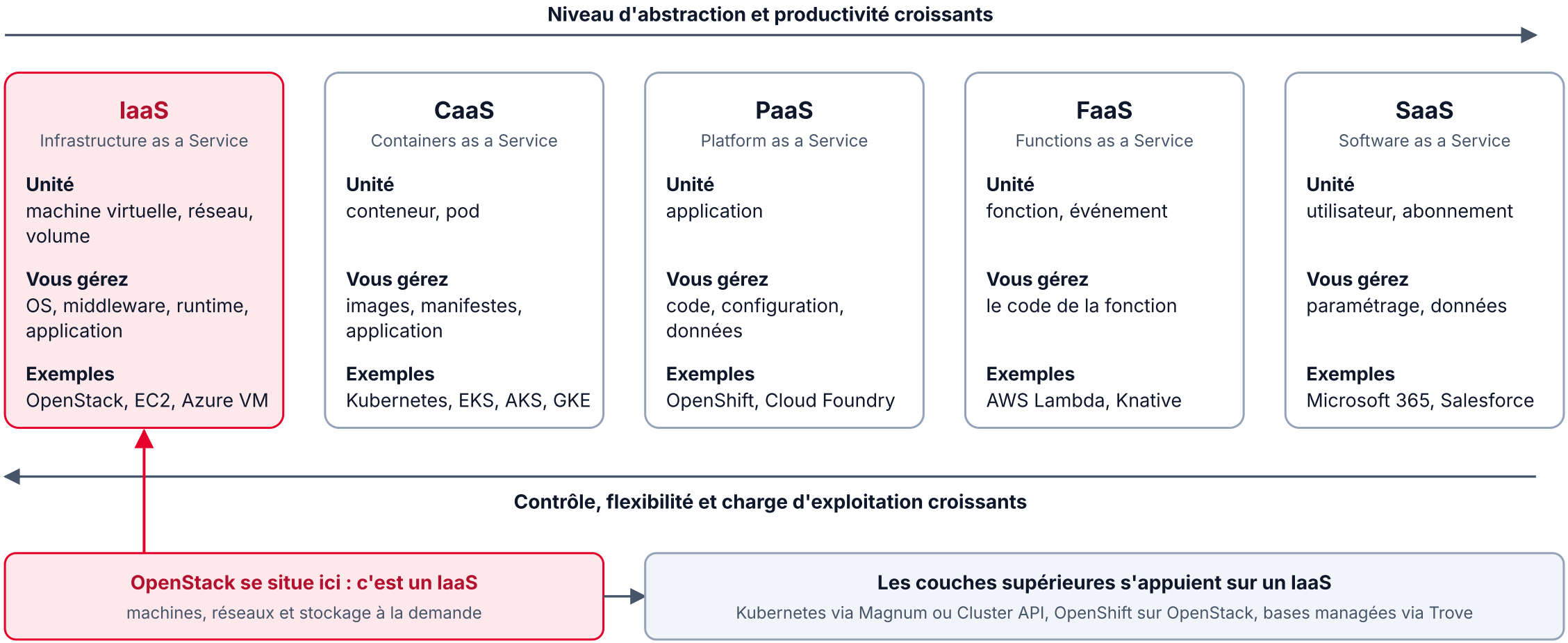
- **Vous gérez :** le code, sa configuration, les données.
- **La plateforme gère :** OS, runtime, certificats, répartition de charge, redémarrages, montées de version.
- **Exemples :** Red Hat OpenShift, Cloud Foundry, Heroku, Azure App Service, Google App Engine.

- **Services managés associés :** bases de données (DBaaS), files de messages, caches — consommés par API, sans administrer de serveur.
- **Atout :** productivité, standardisation, délai de mise en production très court.
- **Contrepartie :** cadre imposé (langages, modèle de déploiement), dépendance à la plateforme.



En dessous : des machines, des réseaux et des volumes fournis par un IaaS — par exemple OpenStack.

IaaS, CaaS, PaaS, FaaS, SaaS : un continuum d'abstraction



Modèles de déploiement : public, privé, hybride



Privé

Infrastructure dédiée à une organisation. Maîtrise totale, investissement et exploitation à sa charge. **Terrain naturel d'OpenStack.**

Public

Ressources mutualisées, ouvertes à tous, facturées à l'usage. Certains clouds publics sont eux-mêmes bâtis sur OpenStack (OVHcloud, Infomaniak).

Hybride

Combinaison d'un privé et d'un ou plusieurs publics, reliés et pilotés de façon cohérente : débordement, reprise d'activité, données sensibles en interne.

Communautaire, multi-cloud

Cloud partagé entre organisations d'un même secteur (recherche, État). Multi-cloud : plusieurs fournisseurs pour limiter la dépendance.

Virtualisation ou cloud : ce qui change vraiment

Un hyperviseur ne fait pas un cloud. La virtualisation est une **technologie** ; le cloud est un **modèle de consommation** construit au-dessus, qui ajoute le libre-service, le multi-tenant et la mesure.

Critère	Virtualisation classique	Cloud (IaaS)
Interface principale	Console d'administration graphique	API REST, CLI, outils d'infrastructure as code
Qui provisionne ?	L'administrateur, sur demande (ticket)	L'utilisateur final, en libre-service, dans la limite de ses quotas
Cloisonnement	Clusters, dossiers, droits d'administration	Projets (tenants) isolés : réseaux, quotas et rôles par projet
Réseau	VLAN créés par l'équipe réseau	Réseaux virtuels créés par l'utilisateur (overlay), routeurs, IP flottantes
Mise à l'échelle	Verticale : on ajoute CPU et RAM à la machine	Horizontale : on ajoute des instances identiques
Disponibilité	Assurée par l'infrastructure (HA, migration à chaud)	Conçue dans l'application (réplication, zones de disponibilité)
Suivi de l'usage	Capacité globale du cluster	Mesure par projet, refacturation possible

« Pets » et « cattle » : l'infrastructure devient du code

	Pets (animaux de compagnie)	Cattle (bétail)
Identité	Serveur nommé, unique	Instance numérotée, interchangeable
En cas de panne	On répare	On détruit et on recrée
Configuration	Manuelle, accumulée au fil du temps	Image + script, reproductible
Évolution	Mise à jour en place	Remplacement par une nouvelle version
État	Sur le disque local	Externalisé : volume, base, objet

- OpenStack sait héberger les deux, mais son modèle natif est le **cattle** : image, gabarit, `cloud-init`, disque éphémère.
- L'état durable vit dans **Cinder** (volumes) et **Swift** (objets), pas dans l'instance.
- Toute l'infrastructure se décrit en fichiers versionnés : Heat, Terraform ou OpenTofu, Ansible.

```
# Une instance complète en une commande, reproductible
openstack server create web-01 \
  --image    ubuntu-24.04 \
  --flavor   m1.small \
  --network net-app \
  --security-group web \
  --key-name cle-deploiement \
  --user-data cloud-init.yaml

# cloud-init.yaml : configuration au premier démarrage
#cloud-config
packages: [nginx]
runcmd:
  - systemctl enable --now nginx
```

> La même requête peut être émise par la CLI, par Horizon, par Terraform ou par un pipeline CI/CD : tous appellent la **même API REST**.

Pourquoi construire un cloud privé ?

Souveraineté et conformité

Localisation et maîtrise juridique des données, exigences sectorielles (défense, santé, finance), qualification SecNumCloud, environnements déconnectés d'Internet.

Maîtrise des coûts

Pour une charge stable et importante, une infrastructure amortie coûte souvent moins cher que la location à l'usage. Pas de frais de sortie de données.

Performance et matériel

Latence maîtrisée, proximité des systèmes existants, matériel spécifique : GPU, SR-IOV, DPDK, bare metal, fonctions réseau télécoms (NFV).

Indépendance

API ouvertes et documentées, code source disponible, pas d'éditeur unique : on peut changer de distribution ou de prestataire sans changer d'API.

Ce que l'on gagne par rapport à la seule virtualisation

- Autonomie des équipes projets : plus de ticket pour une machine ou un réseau.
- Automatisation de bout en bout, intégrable aux chaînes CI/CD.
- Visibilité sur la consommation par projet grâce aux quotas et à la mesure.

Les contreparties à anticiper

- Une **équipe plateforme** compétente : Linux, réseau, stockage, automatisation.
- Des montées de version régulières et une supervision du plan de contrôle.
- Une capacité finie : il faut planifier les achats de matériel.

02

Le projet OpenStack

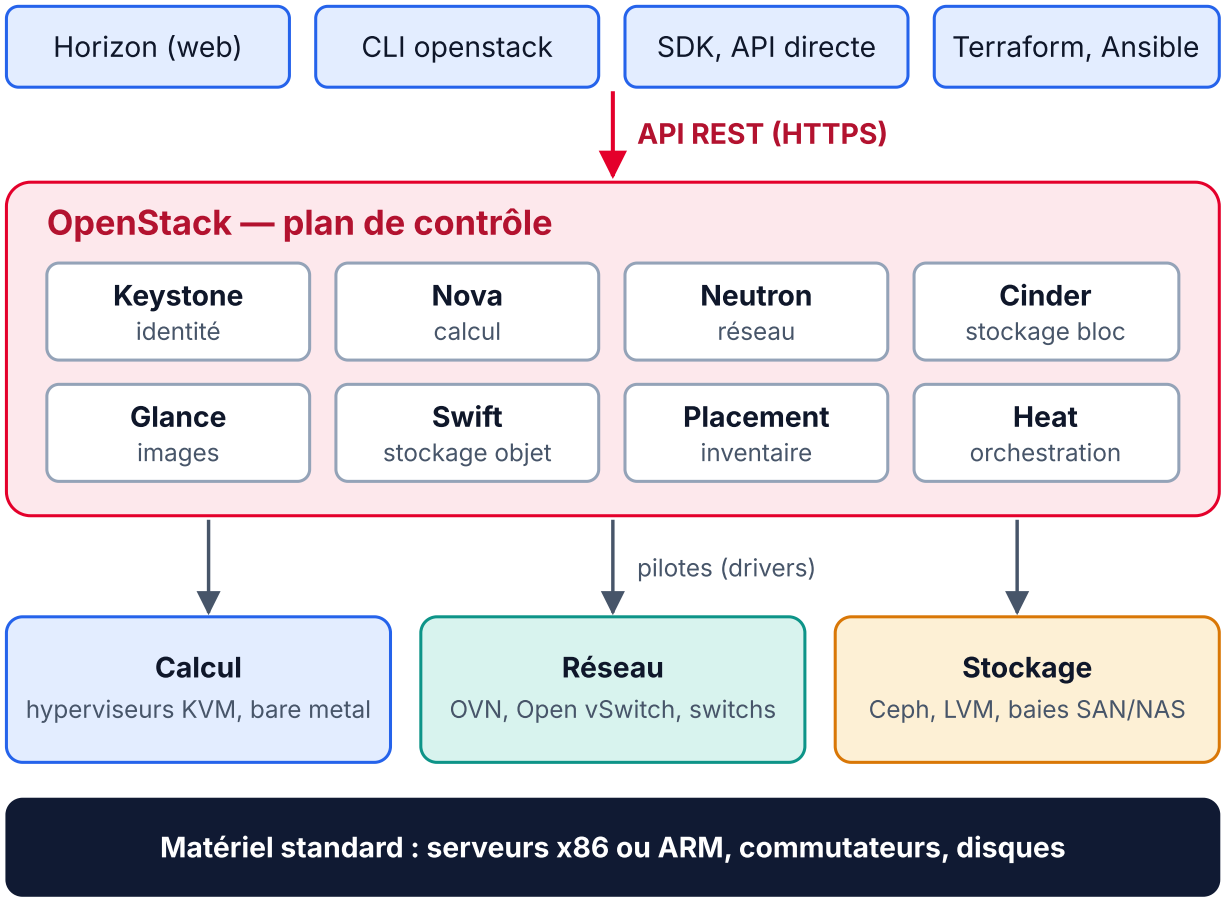
Un logiciel libre né en 2010, devenu l'un des plus grands projets open source au monde : ce qu'il est, qui le pilote, comment il évolue et comment on se le procure.

- Définition : un système d'exploitation de cloud
- Gouvernance ouverte et « Four Opens »
- Qui l'utilise, à quelle échelle
- Seize ans d'histoire
- Cycle de publication et versions SLURP
- Distributions et outils de déploiement

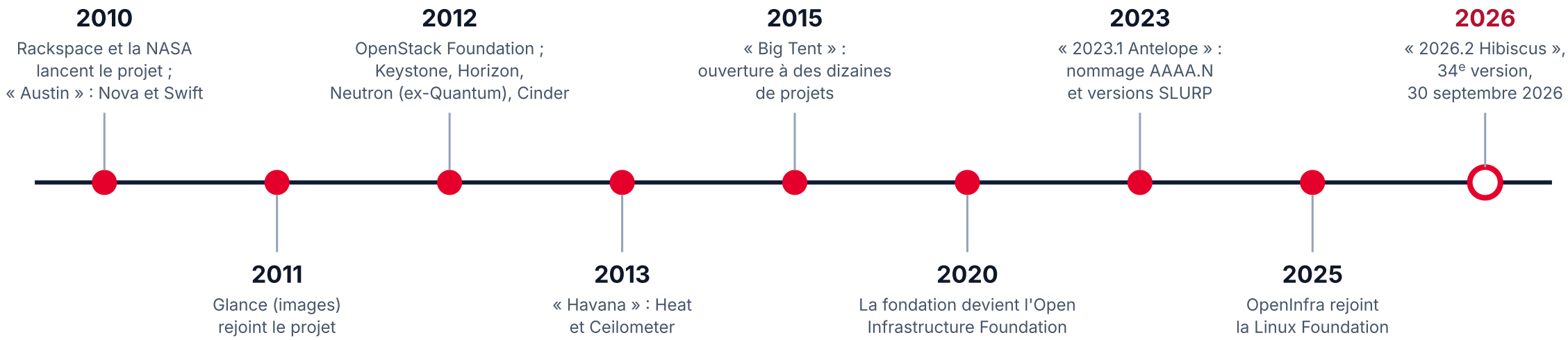
Qu'est-ce qu'OpenStack ?

Un ensemble de **logiciels libres** qui pilote de grands pools de calcul, de réseau et de stockage à l'échelle d'un datacenter, à travers des **API** et un tableau de bord, pour fournir un cloud **IaaS** privé ou public.

- **Licence Apache 2.0**, sans édition « entreprise » fermée : tout le code est public.
- Écrit en **Python**, déployé sur **Linux**, sur du matériel standard.
- **Modulaire :** une trentaine de services indépendants ; on n'installe que ceux dont on a besoin.
- **Agnostique :** il ne virtualise rien lui-même, il **orchestre** des technologies existantes (KVM, Open vSwitch, OVN, Ceph, LVM, baies du marché) grâce à des pilotes.
- Surnommé « **système d'exploitation du cloud** » : comme un OS arbitre CPU, mémoire et disques d'une machine, OpenStack arbitre ceux d'un datacenter entre des projets.



Seize ans d'histoire en neuf jalons



L'origine

La NASA apporte le moteur de calcul (Nova), Rackspace son stockage objet (Swift). L'ambition : un cloud ouvert, sans dépendance à un fournisseur.

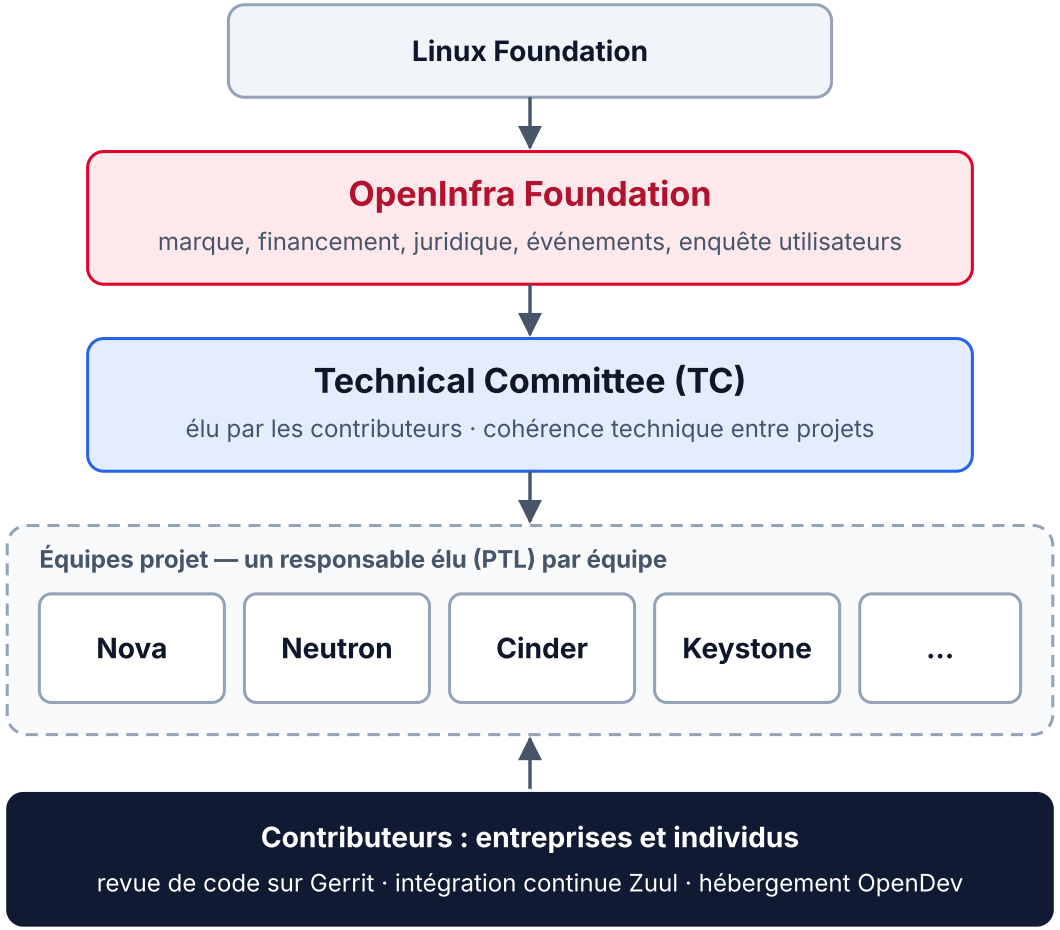
La maturité

Après l'emballement médiatique des années 2015-2017, le projet se recentre sur un cœur stable. Les API principales n'ont plus connu de rupture depuis des années.

Aujourd'hui

Regain d'intérêt porté par la souveraineté numérique, l'évolution des licences VMware et les besoins d'infrastructure pour l'IA (GPU, calcul confidentiel).

Gouvernance : un projet communautaire, sans éditeur unique



Les « Four Opens » — les quatre principes fondateurs :

Open Source

Tout le code sous licence Apache 2.0, sans version réservée aux clients payants.

Open Design

Les orientations se décident en public, lors des PTG (Project Teams Gathering).

Open Development

Chaque modification passe par une revue de code publique et des tests automatiques.

Open Community

Les responsables sont élus ; chacun peut contribuer, quel que soit son employeur.

- Aucune entreprise ne contrôle la feuille de route : les éditeurs (Red Hat, Canonical, Mirantis…) **contribuent** et **distribuent**, ils ne possèdent pas le projet.
- Conséquence pratique : une même API chez tous les fournisseurs, vérifiée par des tests d'interopérabilité.

Cycle de publication : deux versions par an, une SLURP sur deux



Flèches grises : montée de version classique, une version à la fois, tous les six mois.

Cadence

Une version coordonnée tous les **six mois** environ, au printemps et à l'automne. Tous les services sont publiés ensemble et testés ensemble.

Nommage

Depuis 2023 : **année.rang** suivi d'un nom par ordre alphabétique (Antelope, Bobcat, Caracal, Dalmatian, Epoxy, Flamingo, Gazpacho, Hibiscus).

SLURP

Skip Level Upgrade Release Process : les versions **.1** permettent de sauter la version intermédiaire, soit une seule montée de version par an.

Maintenance

Chaque version reçoit des correctifs pendant environ **18 mois** (phase *Maintained*), puis passe en *Unmaintained* avant sa fin de vie.

Qui utilise OpenStack, et à quelle échelle ?

40 millions +

de cœurs de calcul en production recensés dès 2022 par l'enquête utilisateurs de la fondation, en hausse de 60 % sur un an.

300 +

datacenters de cloud public dans le monde reposent sur OpenStack, d'après la même enquête.

> 1 million

de cœurs chez plusieurs organisations à elles seules, parmi lesquelles Walmart, China Mobile, LINE et Workday.

Secteur	Exemples d'utilisateurs	Pourquoi OpenStack
Recherche et calcul	CERN, universités, centres de calcul	Très grande échelle, mutualisation entre équipes, matériel hétérogène, budget maîtrisé
Télécoms	China Mobile, Verizon, grands opérateurs européens	Virtualisation des fonctions réseau (NFV), cœur 4G/5G, performances réseau (SR-IOV, DPDK)
Clouds publics	OVHcloud, Infomaniak, Open Telekom Cloud, Rackspace	Offre IaaS souveraine ou régionale, API standard connue du marché
Distribution et services	Walmart, Workday, LINE, groupe Schwarz (Lidl)	Charges stables et massives, coûts prévisibles, indépendance vis-à-vis des hyperscalers
Secteur public et défense	Administrations, industriels de défense	Souveraineté, code auditable, fonctionnement en réseau isolé

> OpenStack est souvent associé à Kubernetes : la pile **LOKI** (Linux, OpenStack, Kubernetes Infrastructure) est le standard d'infrastructure ouverte mis en avant par la fondation.

Distributions et outils de déploiement

OpenStack « amont » est un ensemble de dépôts de code. Pour l'installer, on passe par un **outil de déploiement** communautaire ou par la **distribution** d'un éditeur, qui ajoute intégration, cycle de vie et support.

Solution	Origine	Principe technique	Usage typique
DevStack	Communauté	Scripts shell, installation depuis les sources sur une seule machine	Développement, découverte — jamais en production
Kolla-Ansible	Communauté	Un conteneur par service (Docker ou Podman), orchestré par Ansible	Production, de quelques nœuds à plusieurs centaines
OpenStack-Ansible	Communauté	Playbooks Ansible, services en conteneurs LXC ou directement sur l'hôte	Production, forte personnalisation
OpenStack-Helm	Communauté	Charts Helm : le plan de contrôle tourne dans Kubernetes	Production pour équipes maîtrisant Kubernetes
Canonical OpenStack	Canonical	Sunbeam : snaps, Juju et Kubernetes ; MAAS pour le matériel	Du poste de test au cloud multi-nœuds, support Ubuntu Pro
Red Hat OpenStack Services on OpenShift	Red Hat	Plan de contrôle géré par des opérateurs sur OpenShift, computes sur RHEL	Entreprises et opérateurs déjà clients Red Hat
Mirantis OpenStack for Kubernetes	Mirantis	OpenStack conteneurisé, cycle de vie piloté par Kubernetes	Clouds privés et opérateurs

Tendance de fond : le plan de contrôle est désormais **conteneurisé**, et de plus en plus souvent hébergé sur Kubernetes. Les API exposées aux utilisateurs restent strictement les mêmes quelle que soit la distribution.

03

OpenStack face aux alternatives

VMware, OpenShift, clouds publics : ces solutions ne répondent pas toutes à la même question. Comparer service par service permet de voir où elles se recouvrent et où elles se complètent.

- VMware : couches et fonctions
- OpenShift : un autre étage de la pile
- Clouds publics : équivalences de services
- VMware : deux philosophies
- OpenStack et Kubernetes, complémentaires
- Grille de choix

OpenStack et VMware : architectures comparées, couche par couche

	VMware Cloud Foundation		OpenStack	
Libre-service ce que voit l'utilisateur	**VCF Automation · vSphere Client** catalogue de services, console d'administration	≈	**Horizon · Skyline · CLI · API REST** même API pour l'humain et pour l'automate	
Plan de contrôle décisions, état, API	**vCenter Server** inventaire, SSO, DRS, HA, modèles de machines	≈	**Keystone**	**Nova** · **Placement** · **Glance**
Réseau et stockage virtualisés, programmables	**NSX** réseaux logiques, pare-feu · **vSAN · VMFS** datastores	≈	**Neutron** OVN, Open vSwitch	**Cinder** Ceph, LVM, baies du marché
Hyperviseur exécute les machines	**ESXi** hyperviseur propriétaire	≈	**KVM / QEMU + libvirt** hyperviseur libre, intégré à Linux	
Matériel serveurs, disques, switchs	**Serveurs certifiés** liste de compatibilité de l'éditeur	≈	**Matériel standard** multi-constructeurs, x86 ou ARM	

> Les couches se correspondent presque une à une. La différence tient au **découpage** : un produit intégré d'un côté, des services indépendants reliés par des API de l'autre.

OpenStack et VMware : au-delà des couches de base

Fonction	OpenStack	VMware (vSphere / VCF)	À savoir
Modèles de VM	**Glance**	Templates, Content Library	Images qcow2 ou raw, préparées pour cloud-init
Identité	**Keystone**	vCenter SSO	Domaines, projets, rôles ; fédération SAML / OIDC
Orchestration	**Heat, Terraform**	VCF Automation, Terraform	Infrastructure décrite en fichiers versionnés
Migration à chaud	**Nova live migration**	vMotion	Déclenchée par l'administrateur ou par un outil
Équilibrage des hôtes	**Watcher**	DRS	Service optionnel dans OpenStack
Redémarrage après panne	**Masakari**	vSphere HA	Service optionnel ; l'approche cloud privilégie la redondance applicative
Répartition de charge	**Octavia**	NSX Advanced Load Balancer (Avi)	Load balancer à la demande, par projet
Kubernetes	**Magnum, Cluster API**	vSphere Kubernetes Service	Clusters créés par API sur l'IaaS

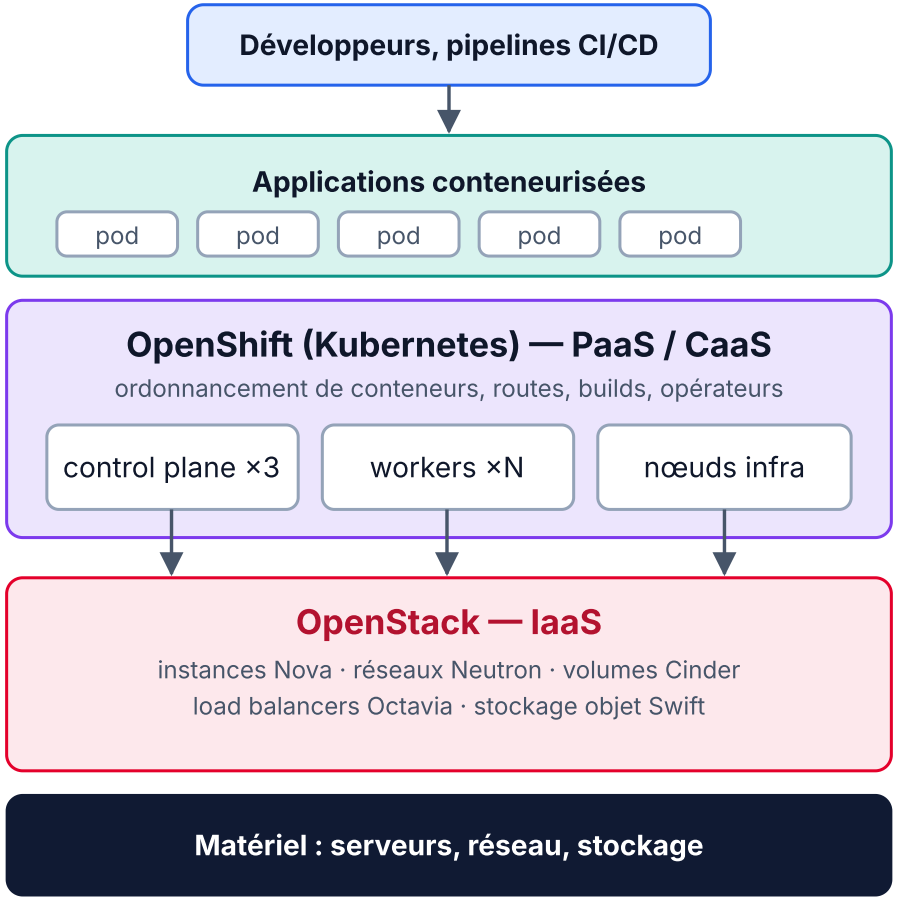
> Les cinq couches de base — hyperviseur, gestion du calcul, réseau, stockage, libre-service — sont mises en regard sur la slide précédente. Ce tableau ne reprend que les fonctions qui s'y ajoutent.

OpenStack et VMware : deux philosophies

Dimension	VMware vSphere / Cloud Foundation	OpenStack
Nature	Produit propriétaire intégré, conçu par un éditeur unique	Projet libre modulaire, porté par une communauté et plusieurs distributeurs
Modèle économique	Abonnement par cœur ; depuis le rachat par Broadcom (fin 2023), fin des licences perpétuelles et offres regroupées en bundles	Aucune licence ; le coût porte sur les compétences, l'exploitation et un éventuel contrat de support
Public visé	Administrateurs d'infrastructure qui opèrent des machines pour le compte des métiers	Équipes projets autonomes qui consomment par API ; l'équipe plateforme opère le cloud
Charges de travail	Applications traditionnelles, machines durables, haute disponibilité transparente	Applications conçues pour le cloud, instances jetables, automatisation ; sait aussi héberger du traditionnel
Matériel	Liste de compatibilité matérielle à respecter	Matériel standard, multi-constructeurs ; choix libre du stockage et du réseau
Exploitation	Expérience intégrée, outillage guidé, compétences répandues	Assemblage plus exigeant : Linux, réseau, stockage distribué, automatisation
Multi-tenant	Ajouté par des produits complémentaires	Natif : projets, quotas, réseaux isolés dès l'origine

Migrer de VMware vers OpenStack ne se résume pas à convertir des disques (`virt-v2v` le fait très bien). Il faut aussi revoir les habitudes : réseau à la demande, sauvegardes, haute disponibilité, processus de demande.

OpenStack et OpenShift : deux étages différents de la pile



	OpenStack	OpenShift
Modèle	IaaS	PaaS / CaaS
Unité gérée	Machine virtuelle, réseau, volume	Conteneur, pod, application
Public	Administrateurs système, équipes infrastructure	Développeurs, équipes applicatives
Moteur	Services Python + KVM	Kubernetes + opérateurs
Éditeur	Communauté, multi-distributeurs	Red Hat (amont libre : OKD)
Isolation	Projet, réseau virtuel, hyperviseur	Namespace, NetworkPolicy, noyau partagé

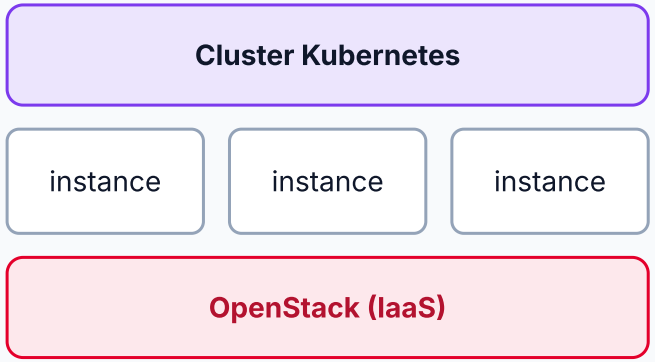
Ce ne sont **pas des concurrents directs** : OpenShift a besoin de machines pour ses nœuds, OpenStack les lui fournit par API.

Zone de recouvrement : OpenShift Virtualization (KubeVirt) exécute des VM dans des pods. Une option pour un parc de VM modeste géré avec les outils Kubernetes ; OpenStack reste plus complet pour un IaaS multi-tenant à grande échelle.

OpenStack et Kubernetes : trois façons de les combiner

A Kubernetes sur OpenStack

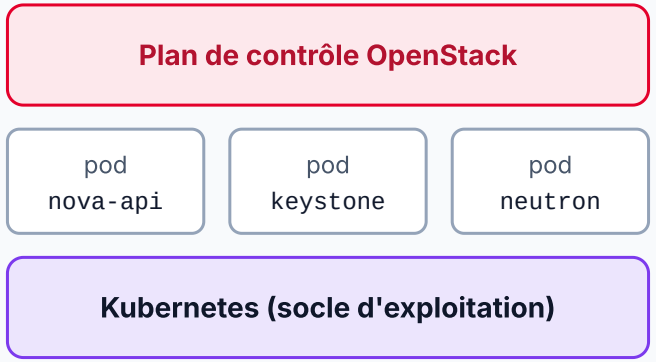
Le cas le plus courant : les clusters tournent dans des instances du cloud.



Création : Magnum ou Cluster API
Volumes : pilote CSI Cinder
Services : LoadBalancer via Octavia
Réseau : ports et sous-réseaux Neutron

B OpenStack sur Kubernetes

Le plan de contrôle d'OpenStack est lui-même déployé comme une application conteneurisée.

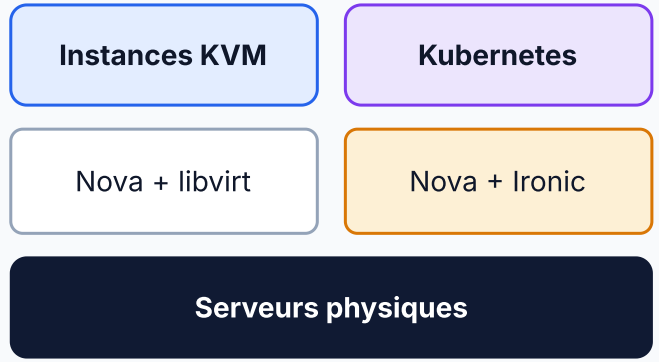


Outils : OpenStack-Helm, Sunbeam, RHOSO, Mirantis MOSK

Intérêt : redémarrage automatique, mises à jour progressives

C Côte à côte sur bare metal

Ironic fournit des serveurs physiques par API ; Kubernetes s'y installe sans hyperviseur.



Usages : GPU, calcul intensif, télécoms, bases de données exigeantes

Intérêt : mêmes API, mêmes réseaux, mêmes quotas pour tout

OpenStack gère l'**infrastructure** (machines, réseaux, disques) ; Kubernetes gère les **applications** (conteneurs). Dans la plupart des clouds privés, on trouve les deux, et souvent les schémas A et B simultanément.

OpenStack et clouds publics : équivalences de services

Fonction	OpenStack	AWS	Microsoft Azure	Google Cloud
Machines virtuelles	**Nova**	EC2	Virtual Machines	Compute Engine
Réseau virtuel	**Neutron**	VPC	Virtual Network	VPC
Stockage bloc	**Cinder**	EBS	Managed Disks	Persistent Disk
Stockage objet	**Swift**	S3	Blob Storage	Cloud Storage
Fichiers partagés	**Manila**	EFS, FSx	Azure Files	Filestore
Images système	**Glance**	AMI	Images, Compute Gallery	Images
Identité et accès	**Keystone**	IAM	Entra ID + RBAC	Cloud IAM
Orchestration	**Heat**	CloudFormation	ARM, Bicep	Infrastructure Manager
Répartition de charge	**Octavia**	Elastic Load Balancing	Load Balancer, Application Gateway	Cloud Load Balancing
DNS	**Designate**	Route 53	Azure DNS	Cloud DNS
Secrets et clés	**Barbican**	KMS, Secrets Manager	Key Vault	Cloud KMS, Secret Manager
Kubernetes managé	**Magnum**	EKS	AKS	GKE
Bases de données	**Trove**	RDS	Azure SQL, Azure Database	Cloud SQL
Métriques et alarmes	**Ceilometer, Aodh**	CloudWatch	Azure Monitor	Cloud Monitoring

> Les concepts sont transposables d'un cloud à l'autre : qui maîtrise Nova, Neutron et Cinder comprend immédiatement EC2, VPC et EBS — et inversement.

OpenStack et clouds publics : deux modèles économiques et opérationnels

Critère	Cloud public (hyperscaler)	OpenStack en cloud privé
Modèle de coût	Dépense d'exploitation, paiement à l'usage, aucun investissement initial	Investissement matériel amorti + coût de l'équipe ; coût marginal faible une fois la capacité en place
Rentabilité	Imbattable pour démarrer, pour les charges variables ou imprévisibles	Avantageuse pour des charges stables, volumineuses et durables
Responsabilité	Le fournisseur exploite tout jusqu'à l'hyperviseur	Vous exploitez tout : datacenter, matériel, plan de contrôle, mises à jour
Catalogue	Des centaines de services managés (bases, IA, analytique, serverless)	IaaS complet et quelques services de plateforme ; le reste se construit dessus
Élasticité	Quasi illimitée, mondiale	Bornée par la capacité achetée ; à planifier
Localisation des données	Régions du fournisseur, droit applicable à examiner	Là où vous le décidez, sous votre contrôle juridique
Dépendance	API propriétaires, frais de sortie des données	API ouvertes, identiques chez tous les fournisseurs OpenStack

Attention au faux débat : OpenStack n'est pas « l'inverse » du cloud public. C'est une technologie, utilisée aussi pour bâtir des clouds publics (OVHcloud, Infomaniak, Open Telekom Cloud).

En pratique : la plupart des organisations sont hybrides. Les outils d'infrastructure as code (Terraform, Ansible) pilotent les deux mondes avec la même démarche.

Synthèse : quelle solution pour quel besoin ?

	OpenStack	VMware Cloud Foundation	OpenShift	Cloud public
Nature	IaaS libre, privé ou public	Virtualisation et cloud privé propriétaires	Plateforme de conteneurs (PaaS / CaaS)	IaaS, PaaS et SaaS loués
Unité de base	Instance, réseau, volume	Machine virtuelle	Conteneur, application	Tout, de la VM à la fonction
Licence	Apache 2.0, gratuite	Abonnement par cœur	Abonnement (OKD libre)	Paiement à l'usage
Compétences clés	Linux, réseau, stockage, automatisation	Administration vSphere	Kubernetes, conteneurs, CI/CD	Architecture cloud, FinOps
Point fort	Ouverture, multi-tenant natif, coût à grande échelle	Intégration, maturité, confort d'exploitation	Productivité des développeurs	Vitesse, étendue du catalogue
Point de vigilance	Exigeant à exploiter	Coût et dépendance à l'éditeur	Nécessite une infrastructure dessous	Coût à l'échelle, souveraineté
Idéal pour	Cloud privé souverain, opérateurs, fournisseurs de cloud	Parc de VM traditionnelles déjà en place	Applications conteneurisées, microservices	Démarrage rapide, charges variables

À retenir : ces solutions se combinent plus qu'elles ne s'excluent. Un schéma très répandu : **OpenStack** pour l'IaaS privé, **Kubernetes ou OpenShift** par-dessus pour les applications, et le **cloud public** en complément pour le débordement ou les services managés.

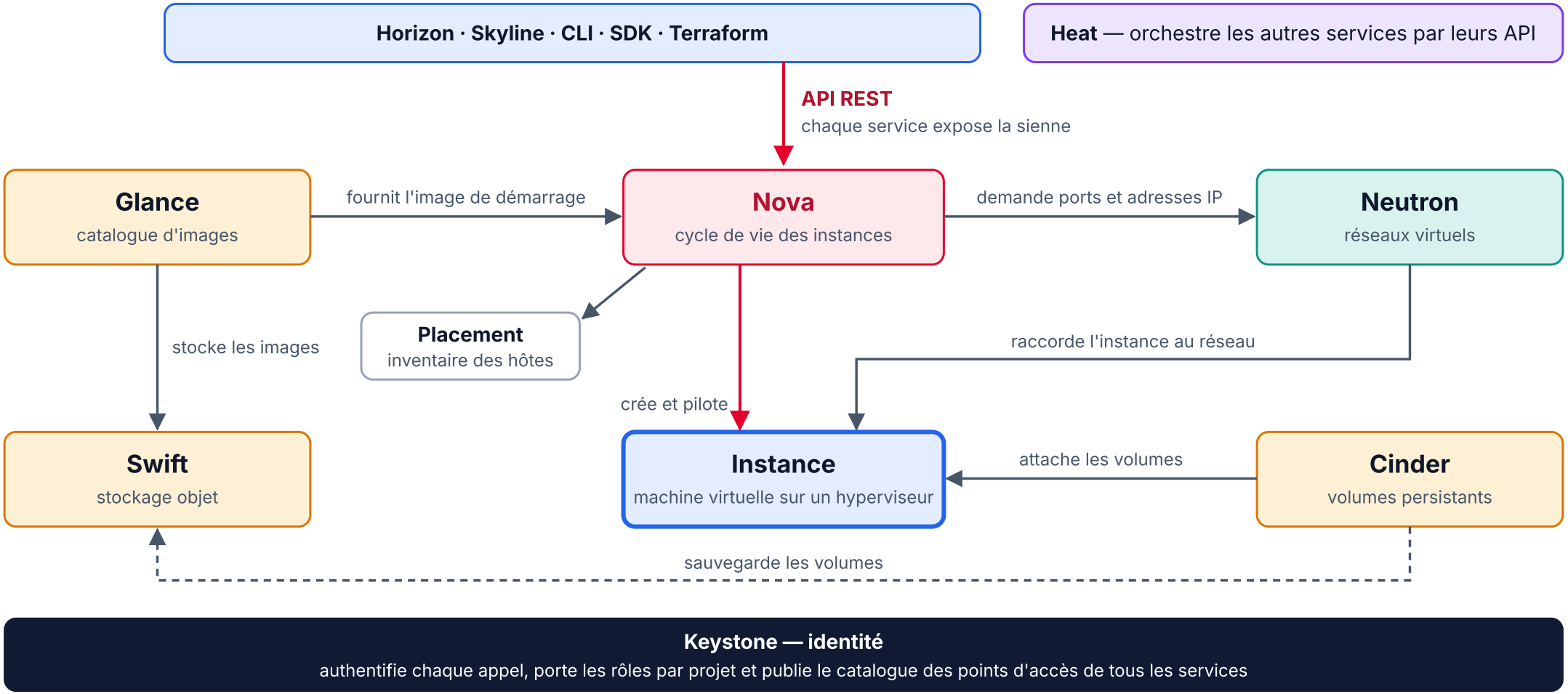
04

Architecture et composants

Comment OpenStack est construit : un socle technique commun, des services spécialisés qui dialoguent par API, et un enchaînement précis lorsqu'un utilisateur demande une ressource.

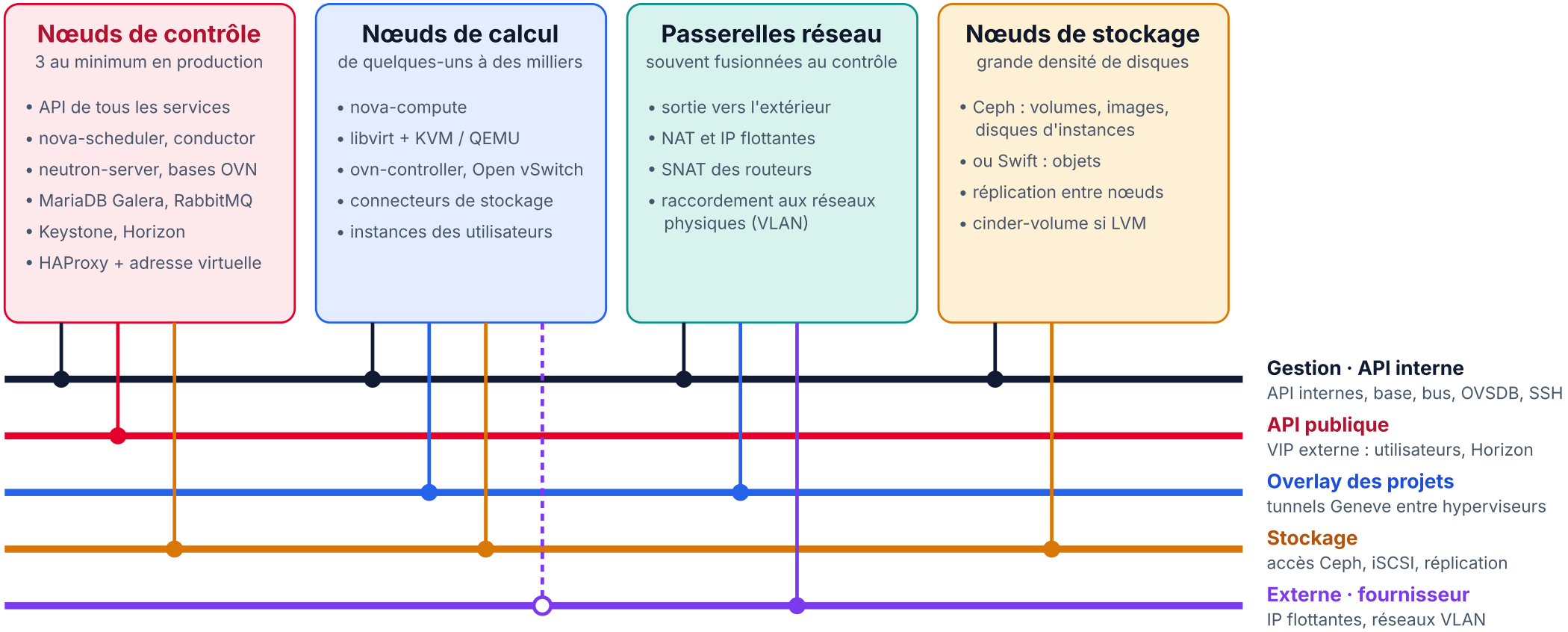
- Vue d'ensemble et types de nœuds
- Keystone, Glance, Nova, Placement
- Horizon, CLI, Heat et services complémentaires
- Socle commun : REST, RPC, base de données
- Neutron, Cinder, Swift
- Création d'une instance, régions et haute disponibilité

Vue d'ensemble : qui fournit quoi à une instance



 Approfondi au module 2

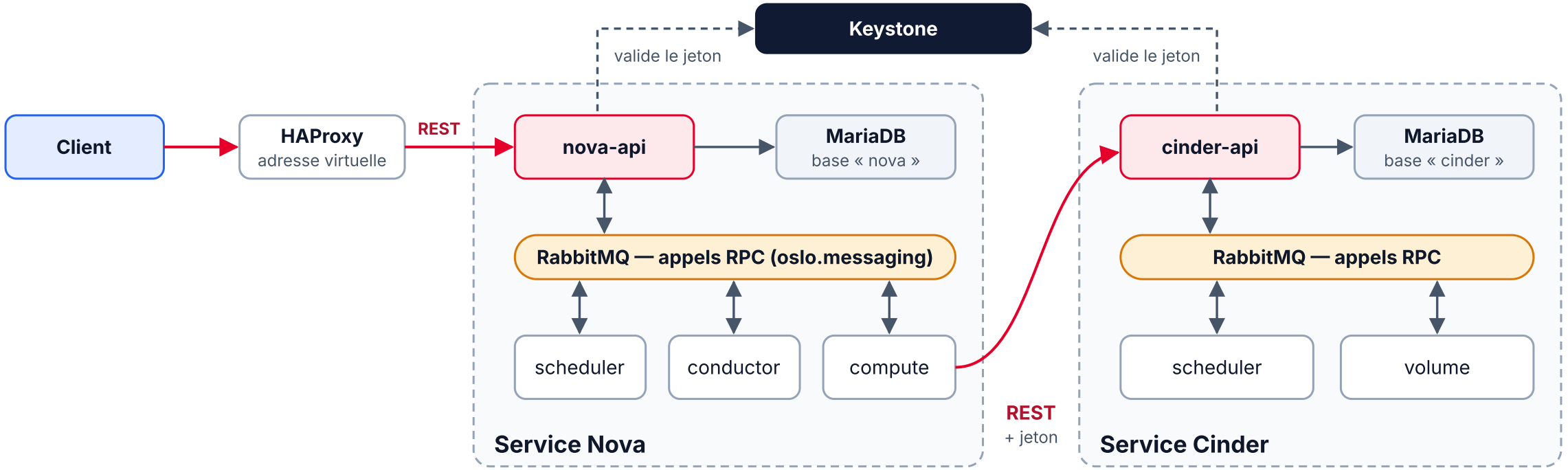
Architecture physique : types de nœuds et réseaux



Séparer les réseaux isole les flux sensibles (gestion, stockage) du trafic des utilisateurs. **Vue simplifiée :** hors bande, provisionnement et réplication du stockage sont détaillés au module 2. Pointillés : le calcul ne rejoint l'externe que si les IP flottantes sont distribuées. Dans un lab, tout peut cohabiter sur une machine.

 Approfondi au module 2

Deux modes de communication : REST entre services, RPC à l'intérieur



Entre services : API REST

Nova ne lit jamais la base de Cinder : il appelle `cinder-api` en HTTP, avec un jeton Keystone, comme le ferait n'importe quel client. Chaque service est donc remplaçable et testable isolément.

Dans un service : RPC asynchrone

Les processus d'un même service échangent par le bus de messages. L'API répond immédiatement (« accepté ») ; le travail se poursuit en arrière-plan, d'où les états transitoires comme `BUILD`.

Un état par service

Chaque service possède sa propre base de données et son propre schéma. Les processus sont sans état : on les multiplie derrière le répartiteur de charge pour la disponibilité et la montée en charge.

4 · ARCHITECTURE ET COMPOSANTS Approfondi au module 3

Le socle technique partagé par tous les services

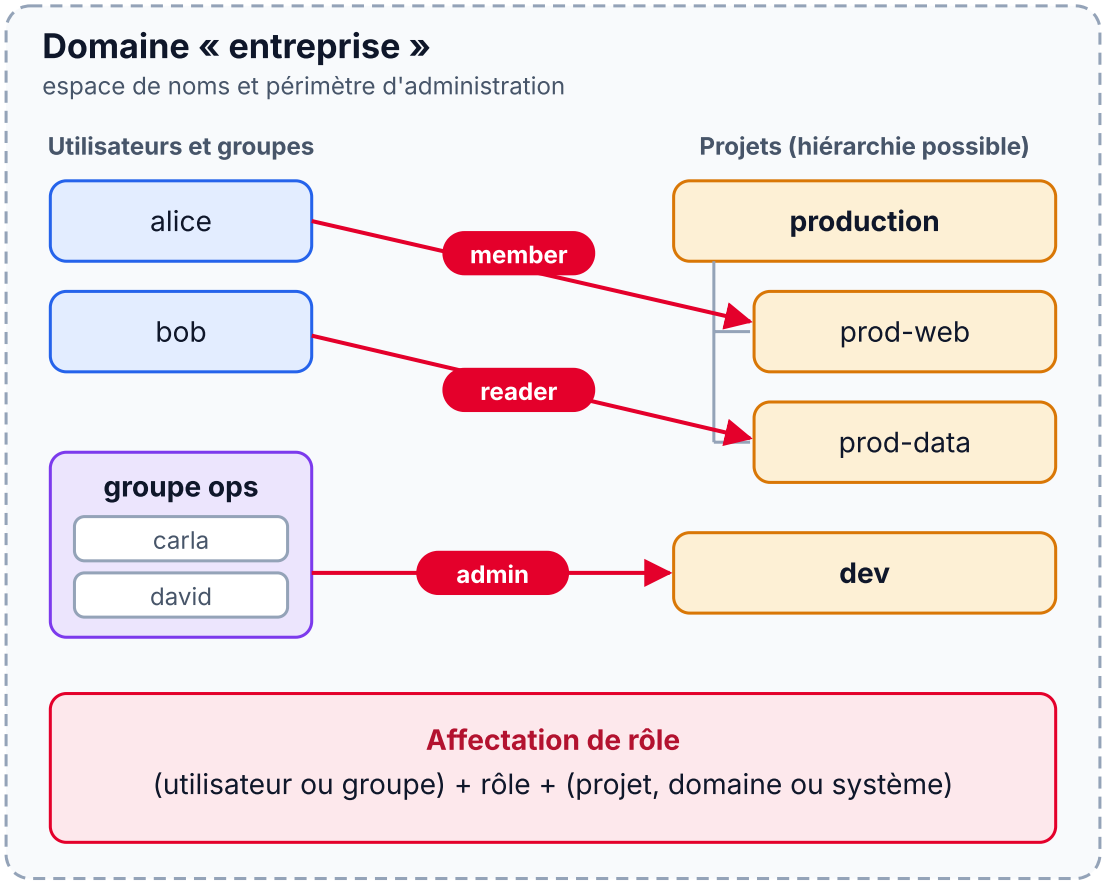
Brique	Technologie de référence	Rôle	Si elle tombe…
Base de données	MariaDB + Galera (3 nœuds)	État de chaque service : instances, réseaux, volumes, utilisateurs	Plus aucune opération d'API possible
Bus de messages	RabbitMQ (files à quorum)	Appels RPC internes et notifications, via la bibliothèque oslo.messaging	Les opérations restent bloquées en cours
Cache	Memcached	Jetons validés, sessions Horizon, résultats fréquents	Ralentissement, charge accrue sur Keystone
Répartiteur de charge	HAProxy + Keepalived ou Pacemaker	Adresse virtuelle unique devant les API, terminaison TLS, bascule	API inaccessibles depuis l'extérieur
Serveur d'applications	Apache mod_wsgi ou uWSGI	Exécute le code Python des API	Le service concerné ne répond plus
Synchronisation horaire	Chrony (NTP)	Horloge commune : validité des jetons, cohérence des clusters	Rejets d'authentification, incohérences

Les bibliothèques Oslo donnent le même comportement à tous les services :

- `oslo.config` : fichiers `/etc/<service>/<service>.conf` au même format partout.
- `oslo.policy` : règles d'autorisation par rôle, surchargeables.
- `oslo.messaging`, `oslo.db`, `oslo.log` : bus, base, journaux.

Point essentiel : une panne du plan de contrôle n'arrête pas les instances. Les machines virtuelles continuent de tourner et de communiquer ; on perd seulement la capacité à créer, modifier ou supprimer des ressources.

Keystone : identité, projets et rôles

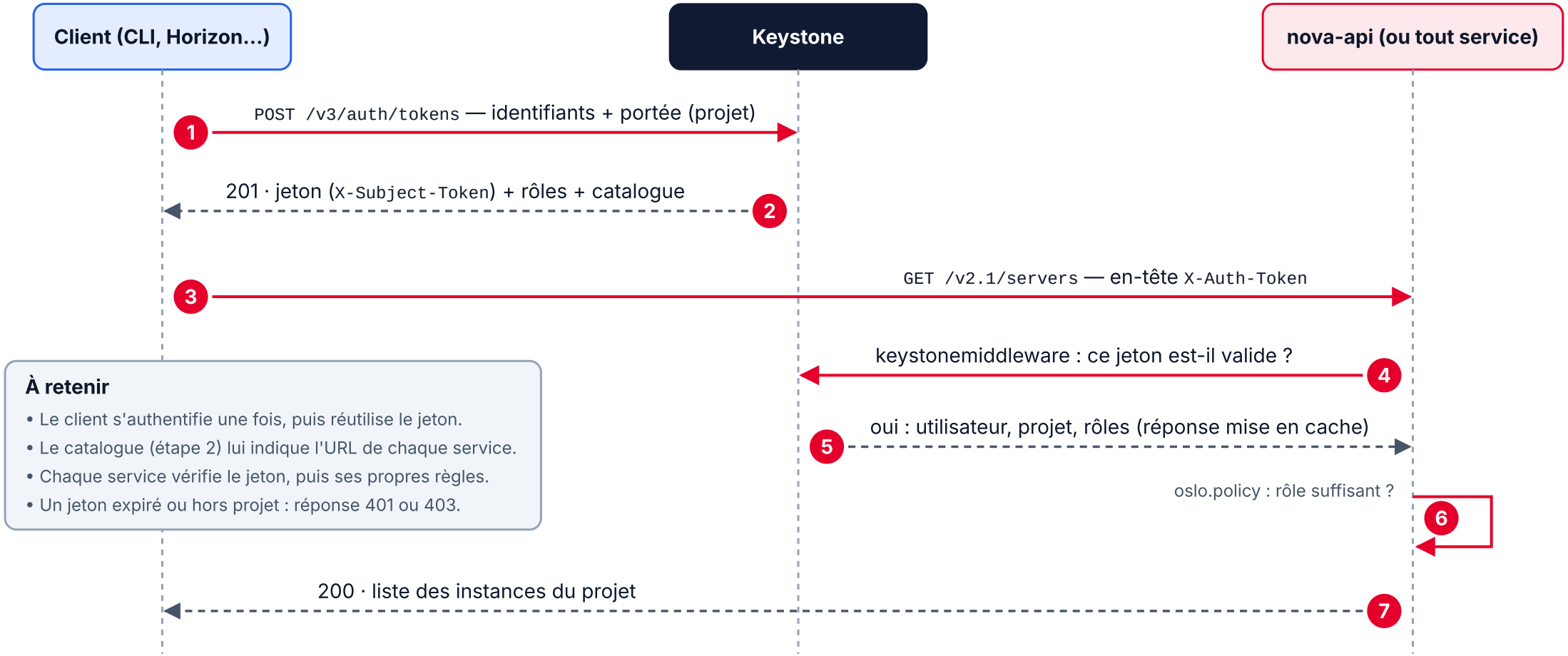


Concept	Définition
Domaine	Conteneur de plus haut niveau : regroupe utilisateurs, groupes et projets ; peut être relié à un annuaire LDAP propre
Projet	Unité d'isolation et de quota (l'ancien « tenant ») : toute ressource appartient à un projet
Rôle	Ensemble de droits ; rôles par défaut : `reader`, `member`, `admin`, chacun incluant le précédent
Jeton	Preuve d'authentification à durée limitée (format Fernet), portée sur un projet
Catalogue	Liste des services et de leurs points d'accès (public, interne, admin) par région
Identifiant d'application	Secret dédié à un automate, limité à un projet, révocable sans toucher au mot de passe

> Keystone est le **premier service installé** et le seul dont tous les autres dépendent. Il peut déléguer l'authentification à un fournisseur externe (SAML, OpenID Connect).

 Approfondi au module 2

Keystone : le parcours d'un jeton

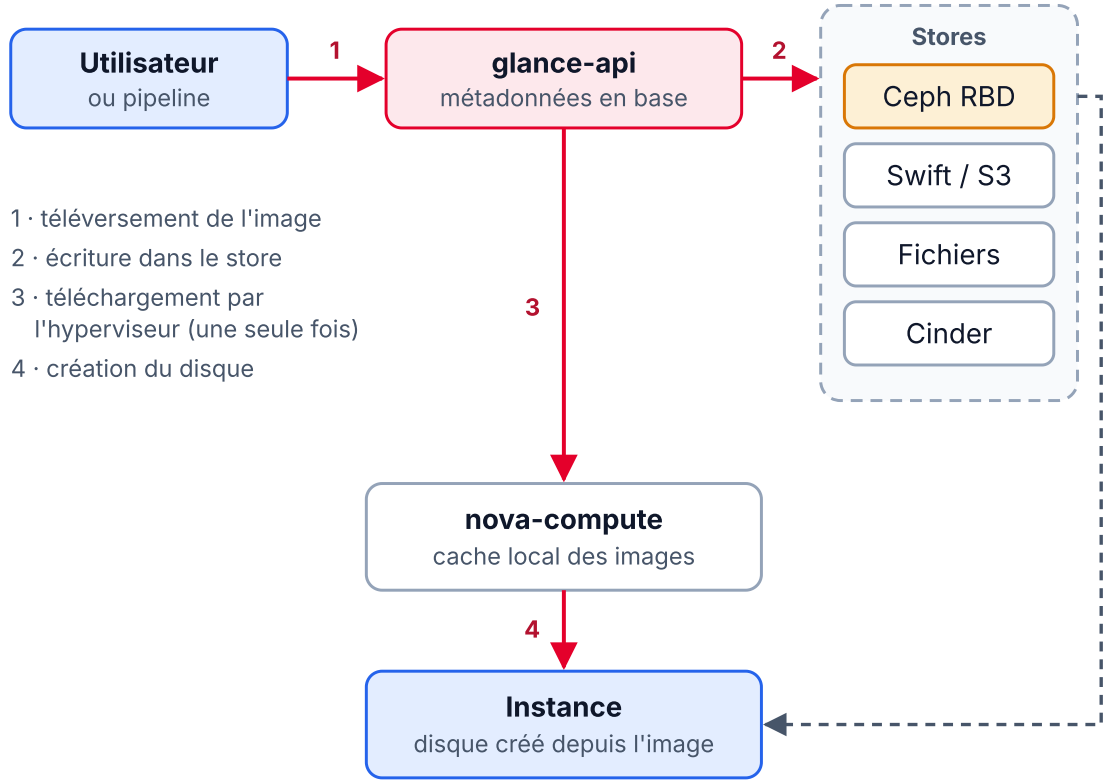


 Approfondi au module 4

Glance : le catalogue d'images

Rôle : enregistrer, décrire et distribuer les **images disque** à partir desquelles les instances démarrent. Glance ne lance rien : il sert de bibliothèque à Nova et à Cinder.

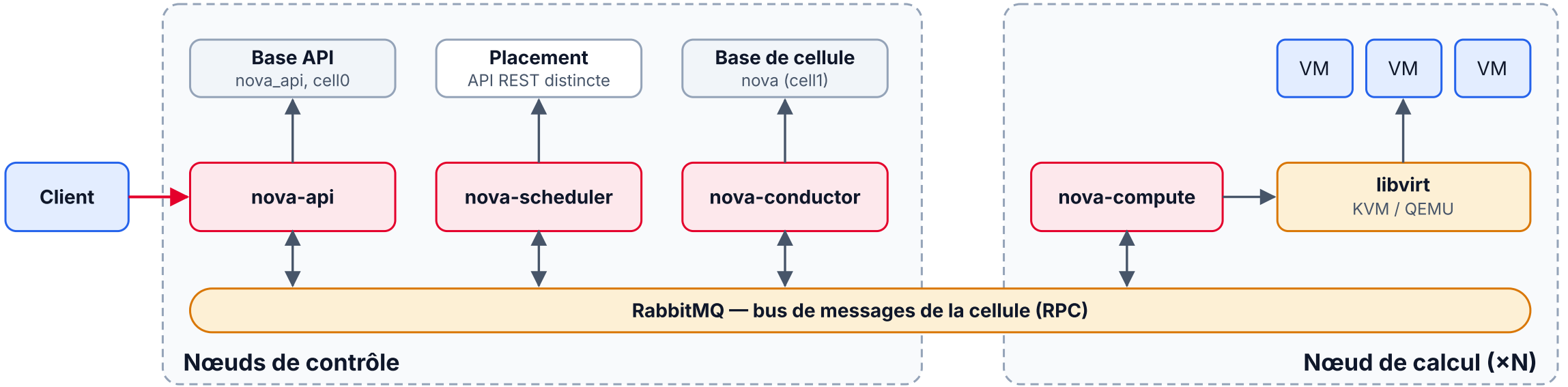
- **Formats :** `qcow2` (compact, le plus courant), `raw` (recommandé avec Ceph), mais aussi `vmdk`, `iso`.
- **Stockage (« stores ») :** système de fichiers, Ceph RBD, Swift, S3, Cinder ; plusieurs stores simultanés possibles.
- **Métadonnées :** propriétés qui guident Nova (`hw_disk_bus`, `hw_firmware_type`, `os_distro`), taille minimale de disque et de RAM.
- **Visibilité :** `public`, `private`, `shared` (partagée avec des projets choisis), `community`.
- **Instantanés :** la capture d'une instance produit une nouvelle image dans Glance.
- **Images « cloud » :** minimales, sans mot de passe, configurées au démarrage par `cloud-init` (clé SSH, nom d'hôte, scripts).



Pointillés : avec Ceph, le disque est un clone instantané de l'image, sans aucun téléchargement (copy-on-write).

 Approfondi au module 4

Nova : l'architecture du service de calcul



nova-api

Reçoit les requêtes REST, vérifie jeton, droits et quotas, enregistre l'instance et répond aussitôt. Point d'entrée unique du service.

nova-scheduler

Choisit l'hôte : interroge Placement pour les candidats, applique des **filtres** puis des **pondérations**.

nova-conductor

Coordonne les opérations longues et fait l'intermédiaire avec la base : les nœuds de calcul n'y accèdent jamais directement.

nova-compute

Un par hyperviseur. Traduit les ordres en appels au pilote : libvirt / KVM (référence), VMware vCenter, Ironic (bare metal).

Cellules (cells v2) : chaque cellule possède sa base et son bus. Ajouter des cellules permet de dépasser plusieurs milliers d'hyperviseurs sans saturer une base unique ; c'est invisible pour l'utilisateur.

 Approfondi au module 4

Nova et Placement : concepts et choix de l'hôte

Concept Nova	Définition
Instance (server)	Machine virtuelle, ou serveur physique avec Ironic
Gabarit (flavor)	Profil de ressources : vCPU, RAM, disque, et propriétés (GPU, CPU dédiés, pages larges)
Paire de clés	Clé SSH publique injectée au premier démarrage
Groupe de serveurs	Règle d'affinité ou d'anti-affinité entre instances
Disque éphémère	Disque local lié à la vie de l'instance : il disparaît avec elle
Opérations	Arrêt, redimensionnement, migration à chaud, évacuation, mise en veille (shelve), secours (rescue), instantané

Placement tient la comptabilité des ressources du cloud, indépendamment de Nova :

- **Fournisseurs de ressources :** chaque hyperviseur, pool de stockage partagé ou carte GPU.
- **Inventaires :** quantités par classe (`VCPU` , `MEMORY_MB` , `DISK_GB` , `VGPU` ...), avec ratio de surallocation.
- **Caractéristiques (traits) :** capacités qualitatives, par exemple `HW_CPU_X86_AVX2` ou `COMPUTE_STATUS_DISABLED` .
- **Allocations :** ce que chaque instance consomme, et sur quel fournisseur.

> La surallocation (plus de vCPU promis que de cœurs physiques) se règle par hôte : `cpu_allocation_ratio` , `ram_allocation_ratio` .



4 · ARCHITECTURE ET COMPOSANTS Approfondi au module 4

Neutron : les objets du réseau virtuel

Rôle : offrir le « réseau en libre-service ». Chaque projet crée ses propres réseaux, routeurs et règles de filtrage, isolés de ceux des autres, sans intervention de l'équipe réseau.

Objet	Définition	Équivalent dans un réseau physique
Réseau (network)	Domaine de diffusion de niveau 2, isolé ; de type interne au projet ou « fournisseur »	Un VLAN
Sous-réseau (subnet)	Plage d'adresses IPv4 ou IPv6 associée à un réseau, avec passerelle, DHCP et DNS	Un plan d'adressage et son serveur DHCP
Port	Point de raccordement : une adresse MAC, une ou plusieurs IP, des groupes de sécurité	Une prise sur un commutateur
Routeur	Relie des sous-réseaux entre eux et vers l'extérieur ; assure la traduction d'adresses	Un routeur avec NAT
IP flottante	Adresse du réseau externe associée à la demande à un port privé (NAT 1:1)	Une adresse publique redirigée
Groupe de sécurité	Règles de filtrage à état appliquées à un port ; tout trafic entrant est refusé par défaut	Un pare-feu placé devant chaque machine
Réseau fournisseur	Réseau créé par l'administrateur, adossé à un VLAN ou à un réseau physique existant	Le réseau du datacenter

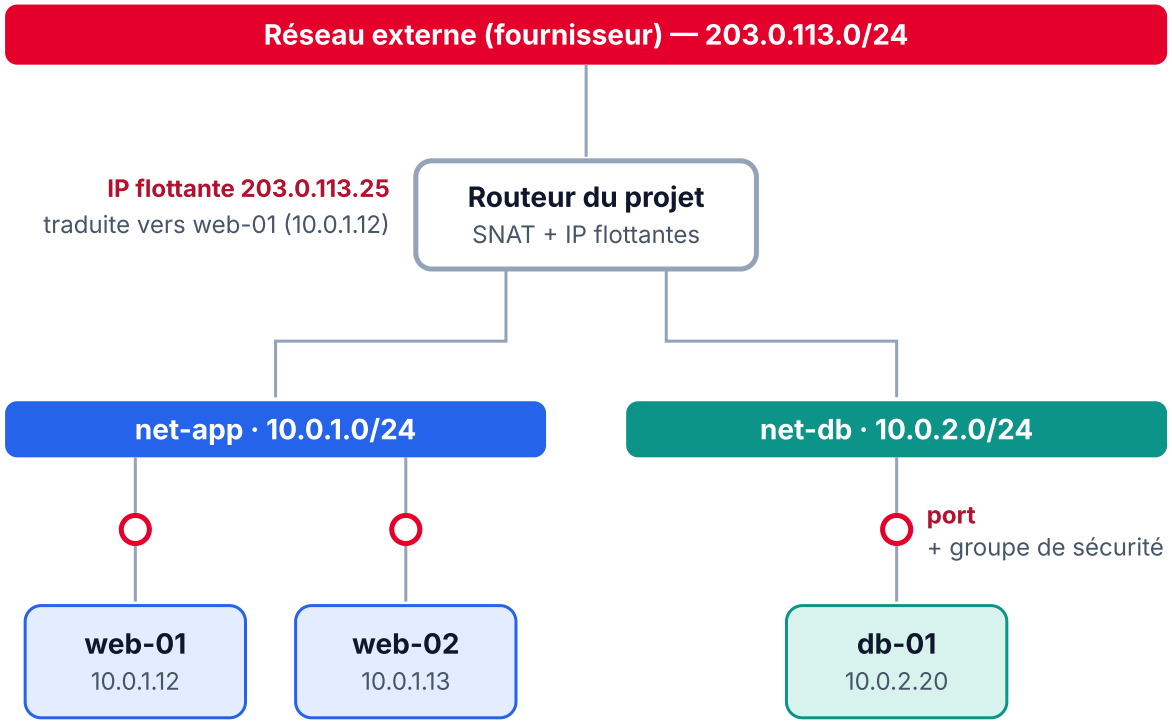
Types de segmentation : `flat` et `vlan` s'appuient sur le réseau physique ; `geneve` et `vxlan` encapsulent le trafic des projets dans des tunnels entre hyperviseurs (overlay).

Services additionnels : DHCP, DNS interne, service de métadonnées (`169.254.169.254`), qualité de service, trunks VLAN, VPN, BGP.

 Approfondi aux modules 2 et 4

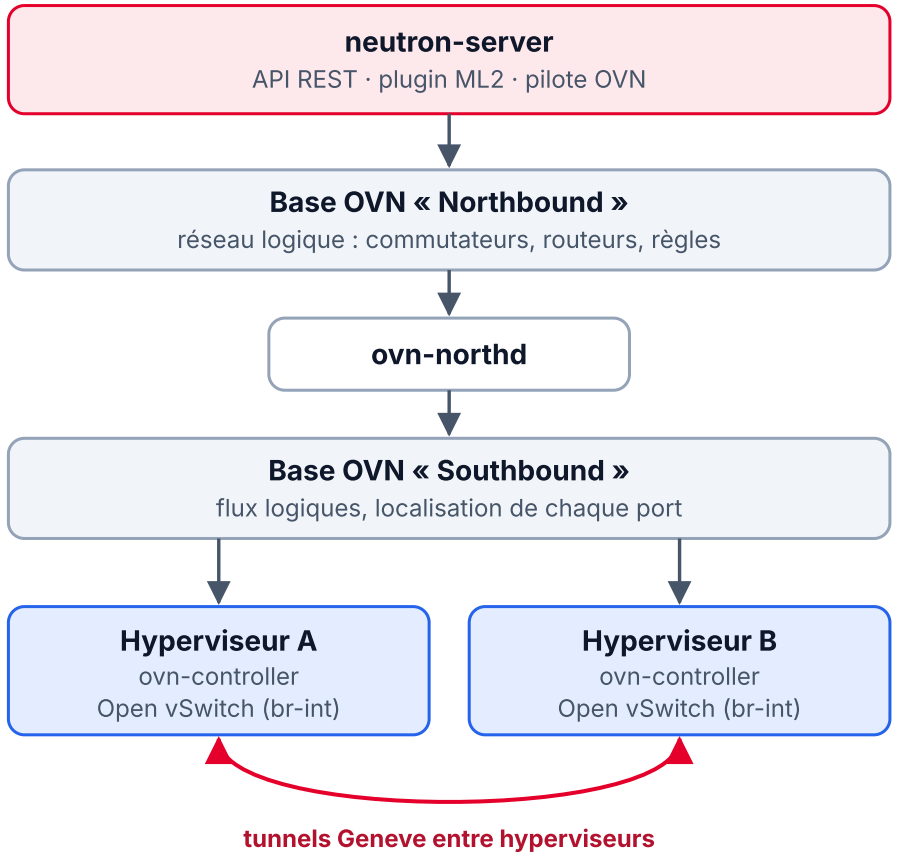
Neutron : topologie d'un projet et mise en œuvre avec OVN

Vue logique : ce que crée l'utilisateur



- Deux réseaux d'un même projet ne communiquent qu'à travers un routeur.
- Deux projets peuvent utiliser les mêmes plages d'adresses sans conflit.
- Sans IP flottante, une instance sort par le routeur (SNAT) mais n'est pas joignable de l'extérieur.

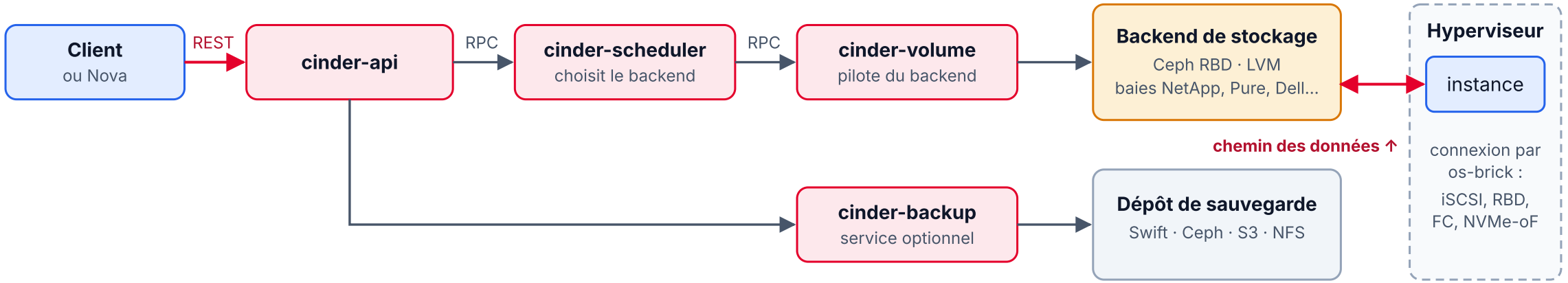
Mise en œuvre de référence : ML2 + OVN



Avec OVN, routage, DHCP, NAT et groupes de sécurité sont **distribués** sur chaque hyperviseur, sans agent dédié. L'ancienne mise en œuvre (ML2 + Open vSwitch avec agents L3, DHCP et métadonnées) reste répandue dans les clouds existants.

 Approfondi au module 4

Cinder : le stockage bloc persistant

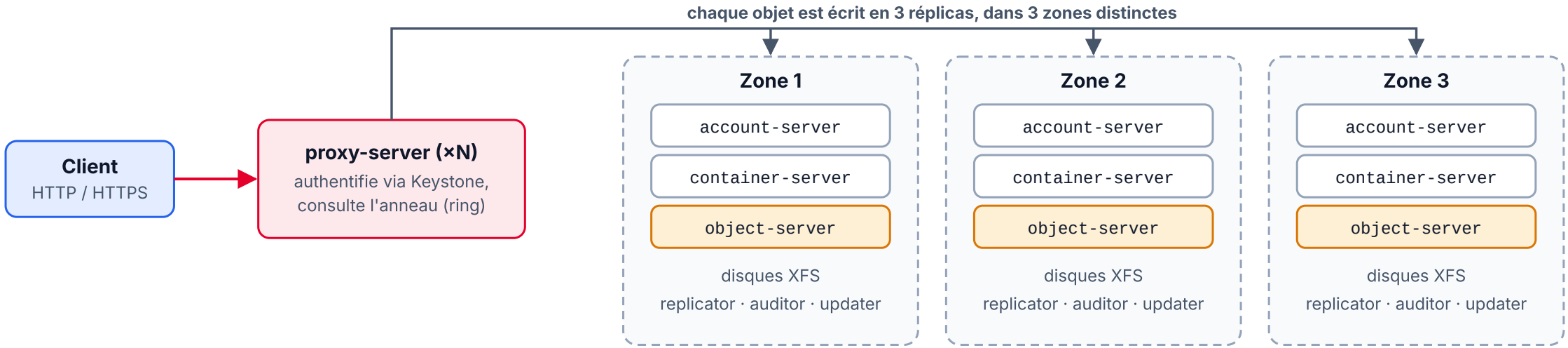


Concept	Définition
Volume	Disque virtuel persistant, attaché à une instance, indépendant de sa durée de vie
Type de volume	Classe de service : backend visé, performances (QoS), chiffrement, réplication
Instantané	Image d'un volume à un instant donné, conservée sur le même backend
Sauvegarde	Copie complète ou incrémentale vers un dépôt distinct
Démarrage sur volume	Le disque système est un volume : l'instance survit à la perte de son hôte

Cinder est hors du chemin des données. Il crée le volume et transmet les informations de connexion ; ensuite l'hyperviseur dialogue directement avec le backend. Un arrêt de Cinder n'interrompt pas les E/S en cours.

- Opérations : extension à chaud, changement de type, migration entre backends, transfert entre projets.
- Attachement multiple possible pour les applications en cluster.

Swift : le stockage objet distribué



Concept	Définition
Compte	Espace de stockage d'un projet Keystone
Conteneur	Regroupement d'objets (équivalent d'un bucket S3) ; porte les droits d'accès
Objet	Donnée + métadonnées, lue et écrite en entier par HTTP
Anneau (ring)	Table qui associe chaque objet à ses disques par hachage du nom

- Adresse d'un objet : `/v1/AUTH_<projet>/<conteneur>/<objet>`.
- **Aucun point central :** pas de base de données maîtresse ; la capacité croît en ajoutant des nœuds.
- **Cohérence à terme :** la disponibilité prime ; les réplicas convergent en arrière-plan.
- Réplication ×3 ou **codage à effacement** pour réduire le coût au Go.
- API compatible **S3** via le middleware `s3api`.
- Alternative fréquente : **Ceph RGW**, qui expose les mêmes API Swift et S3.

Quatre types de stockage, quatre usages

	Éphémère	Bloc	Objet	Fichiers partagés
Service	Nova	Cinder	Swift	Manila
Ce que voit l'utilisateur	Le disque de l'instance	Un disque supplémentaire (`/dev/vdb`)	Une URL HTTP	Un partage réseau à monter
Durée de vie	Celle de l'instance	Indépendante, jusqu'à suppression	Indépendante	Indépendante
Accès	Une seule instance	Une instance (plusieurs en option)	Tout client HTTP, de partout	Plusieurs instances en même temps
Protocole	Disque virtuel local ou Ceph	iSCSI, RBD, Fibre Channel, NVMe-oF	API REST Swift ou S3	NFS, CephFS, SMB
Modification	Lecture et écriture par blocs	Lecture et écriture par blocs	Objet remplacé en entier	Lecture et écriture par fichiers
Usages types	Système, cache, données temporaires	Bases de données, disque système durable	Sauvegardes, médias, journaux, images	Répertoires communs, applications existantes
Équivalent AWS	Instance store	EBS	S3	EFS

Piège classique : supprimer une instance supprime son disque éphémère. Toute donnée à conserver doit être placée sur un volume Cinder, dans Swift ou sur un partage Manila.

Ceph, le backend unificateur : un même cluster Ceph peut servir Glance (images), Cinder (volumes), Nova (disques éphémères), Manila (CephFS) et l'API objet (RGW).

Accéder au cloud : Horizon, CLI, SDK et outils d'automatisation

Interface	Description
Horizon	Tableau de bord web historique (Django) : vues « Projet », « Admin », « Identité »
Skyline	Tableau de bord moderne, plus réactif, alternative à Horizon
CLI `openstack`	Client unifié (python-openstackclient) : une commande pour tous les services
openstacksdk	Bibliothèque Python officielle ; Gophercloud pour Go
Terraform, OpenTofu	Fournisseur `openstack` : infrastructure déclarative et versionnée
Ansible	Collection `openstack.cloud` : modules pour chaque ressource

- Toutes ces interfaces sont de simples **clients de la même API REST** : ce qu'Horizon sait faire, un script sait le faire.
- Les **microversions** permettent à l'API d'évoluer sans casser les anciens clients : le client annonce la version qu'il comprend.
- L'option `--debug` de la CLI affiche chaque appel HTTP : idéale pour comprendre les échanges entre services.

```
# ~/.config/openstack/clouds.yaml
clouds:
  formation:
    auth_type: v3applicationcredential
    auth:
      auth_url: https://cloud.example.org:5000/v3
      application_credential_id: 7f3c2a91d4e84b6fa0c5
      application_credential_secret: Zq8vT2mLx4RkPw9s
    region_name: RegionOne
    interface: public
```

```
export OS_CLOUD=formation

openstack catalog list          # services et points d'accès
openstack server list           # instances du projet
openstack network create net-app
openstack --debug image list    # détaille les appels REST
```

Heat : l'orchestration par modèles

Rôle : créer, mettre à jour et supprimer un ensemble cohérent de ressources — une **pile** (stack) — décrit dans un **modèle** texte au format HOT (YAML).

- **Déclaratif :** on décrit l'état voulu ; Heat calcule l'ordre de création à partir des dépendances entre ressources.
- **Types de ressources :** `OS::Nova::Server`, `OS::Neutron::Net`, `OS::Cinder::Volume`, `OS::Octavia::LoadBalancer`…
- **Paramètres et sorties :** un même modèle sert à plusieurs environnements.
- **Mise à l'échelle automatique :** groupes d'instances pilotés par des alarmes Aodh.
- **Architecture :** `heat-api` reçoit le modèle, `heat-engine` appelle les API des autres services.

> **Heat ou Terraform ?** Heat est intégré et s'exécute côté cloud ; Terraform et OpenTofu sont multi-clouds et dominent aujourd'hui côté utilisateurs. Heat reste très utilisé par les services internes, notamment Magnum.

```yaml
heat_template_version: 2021-04-16
parameters:
  image: {type: string, default: ubuntu-24.04}
resources:
  reseau:
    type: OS::Neutron::Net
  sous_reseau:
    type: OS::Neutron::Subnet
    properties:
      network: {get_resource: reseau}
      cidr: 10.0.1.0/24
  serveur:
    type: OS::Nova::Server
    depends_on: sous_reseau
    properties:
      image: {get_param: image}
      flavor: m1.small
      networks:
        - network: {get_resource: reseau}
outputs:
  adresse:
    value: {get_attr: [serveur, first_address]}
```

```
openstack stack create -t pile.yaml demo
```

Les services complémentaires

Octavia
Répartition de charge à la demande : écouteurs, pools, sondes de santé. S'appuie sur des instances dédiées (amphores) ou sur OVN.

Designate
DNS en libre-service : zones et enregistrements par projet, création automatique d'entrées pour les instances et les IP flottantes.

Barbican
Coffre-fort : secrets, clés de chiffrement et certificats, utilisés par Cinder (volumes chiffrés), Octavia (TLS) et Nova.

Manila
Systèmes de fichiers partagés (NFS, CephFS, SMB) montables par plusieurs instances simultanément.

Magnum
Clusters Kubernetes à la demande, créés par API sur des instances du projet ; s'appuie sur Heat ou Cluster API.

Ironic
Bare metal : provisionne des serveurs physiques (IPMI, Redfish, PXE) avec la même API Nova que pour les machines virtuelles.

Zun
Conteneurs individuels lancés par API, sans cluster Kubernetes à gérer.

Trove
Bases de données managées (MySQL, MariaDB, PostgreSQL) : création, sauvegarde, réplication.

Ceilometer, Gnocchi, Aodh
Télémétrie : collecte des mesures, stockage en séries temporelles, alarmes déclenchant des actions.

CloudKitty
Valorisation de l'usage : transforme les mesures en coûts par projet pour la facturation ou la refacturation interne.

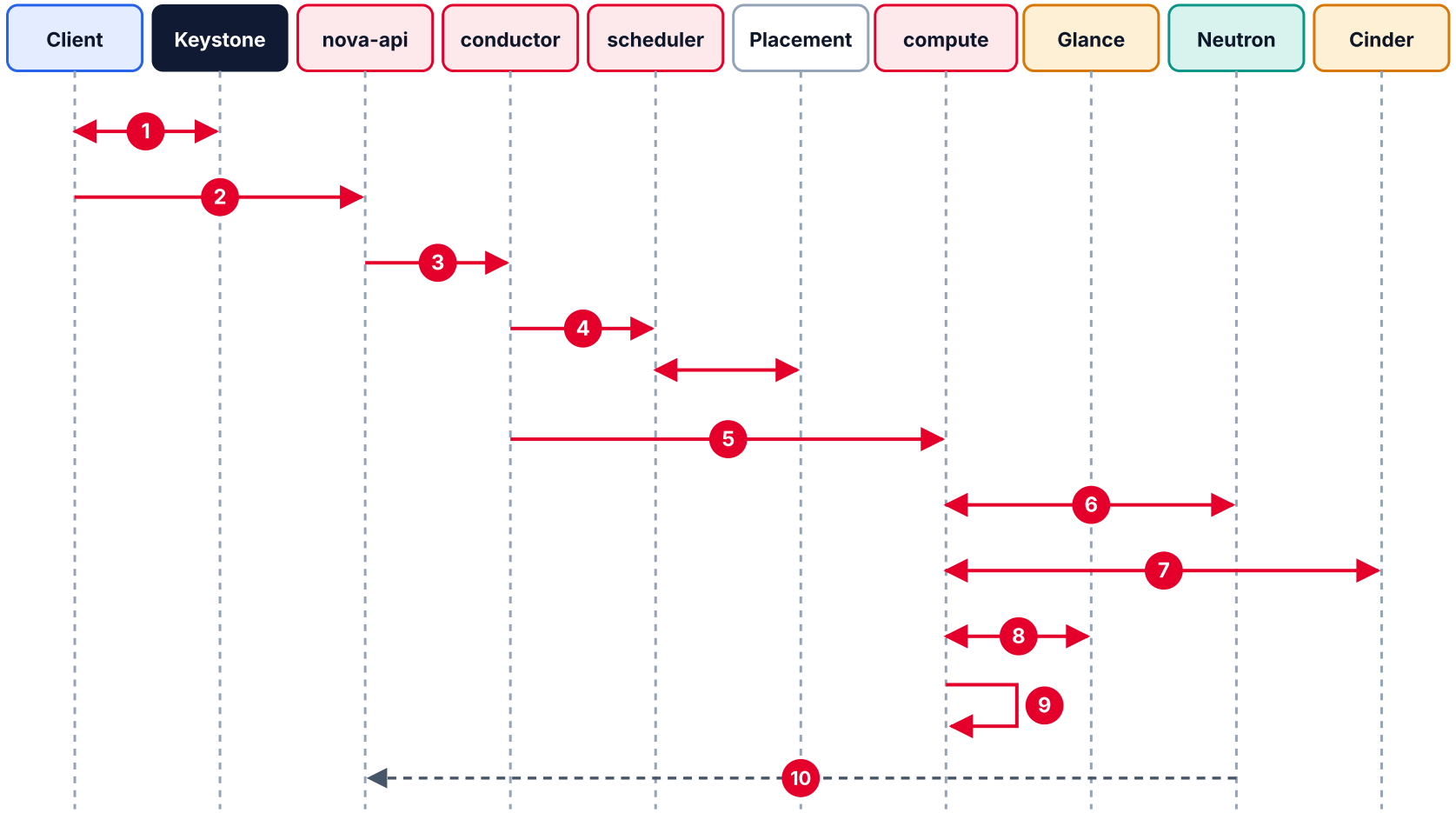
Masakari
Haute disponibilité des instances : détecte la panne d'un hôte ou d'une VM et relance ailleurs.

Watcher, Blazar, Cyborg
Optimisation du placement, réservation de ressources à l'avance, gestion des accélérateurs (GPU, FPGA).

> Tous suivent le même patron : une API REST enregistrée dans le catalogue Keystone, une base, des processus reliés par le bus de messages. **On n'active que ce dont on a besoin** : le noyau minimal est Keystone, Glance, Placement, Nova et Neutron.

 Approfondi aux modules 2 et 4

Tout assembler : la création d'une instance, étape par étape



1 Authentification : jeton et catalogue.

2 `POST /servers` : image, gabarit, réseau.

3 Jeton, droits et quotas vérifiés ; instance enregistrée à l'état BUILD ; réponse 202.

4 Le scheduler interroge Placement, filtre, pondère et réserve les ressources.

5 Ordre de construction envoyé à l'hôte retenu, par le bus de messages.

6 Création et liaison du port : adresse MAC, IP, groupes de sécurité.

7 Attachement du volume, en cas de démarrage sur volume.

8 Récupération de l'image (ou clone Ceph).

9 libvirt définit et démarre la machine KVM.

10 Neutron signale que le port est actif : l'instance passe à l'état ACTIVE.

Rouge : appels · pointillés : notification

Une simple création d'instance mobilise **six services** et les deux modes de communication : REST entre services (étapes 1, 2, 4, 6, 7, 8, 10) et RPC à l'intérieur de Nova (étapes 3, 4, 5). C'est le meilleur fil conducteur pour diagnostiquer une erreur.

 Approfondi au module 2

Passer à l'échelle : régions, zones, agrégats, cellules



Niveau	Visible par	À quoi il sert
Région	Utilisateur	Séparer des sites : chaque région a ses propres services et points d'accès
Zone de disponibilité	Utilisateur	Isoler les pannes (salle, alimentation) ; choisie à la création de l'instance
Agrégat d'hôtes	Administrateur	Regrouper des hôtes par caractéristique (GPU, SSD, licence) et y diriger certains gabarits
Cellule	Administrateur	Répartir la charge de la base et du bus pour gérer des milliers d'hôtes

Haute disponibilité du plan de contrôle :

- **Trois nœuds de contrôle** au minimum, pour conserver un quorum.
- API sans état, répliquées derrière **HAProxy** et une adresse virtuelle.
- Base **MariaDB Galera** (réplication synchrone) ; **RabbitMQ** en cluster avec files à quorum.
- Côté données : Ceph répliqué ×3, zones de disponibilité, éventuellement Masakari.

SYNTHÈSE

À retenir

1 · Un cloud, c'est un modèle

Libre-service, accès réseau, ressources mutualisées, élasticité, usage mesuré. La virtualisation seule n'en fait pas un.

2 · OpenStack est un IaaS libre

Il orchestre calcul, réseau et stockage existants par des pilotes, sous licence Apache 2.0, avec une gouvernance communautaire.

3 · Il se combine aux autres

Équivalent ouvert de la pile VMware, socle pour Kubernetes et OpenShift, mêmes concepts que les clouds publics.

4 · Des services indépendants

Chacun a son API REST, sa base et ses processus. REST entre services, RPC par le bus de messages à l'intérieur.

5 · Keystone au centre

Chaque appel porte un jeton lié à un projet ; les rôles décident des droits ; le catalogue indique où joindre chaque service.

6 · Contrôle et données séparés

Une panne du plan de contrôle bloque les opérations, pas les instances, ni leurs réseaux, ni leurs volumes.

Besoin	Service	En une phrase
Qui suis-je, qu'ai-je le droit de faire ?	**Keystone**	Authentifie, autorise par projet, publie le catalogue
Démarrer une machine	**Nova + Placement**	Choisit un hôte et pilote l'hyperviseur
À partir de quel système ?	**Glance**	Conserve et distribue les images
La connecter	**Neutron**	Réseaux, routeurs, IP flottantes, groupes de sécurité
Conserver ses données	**Cinder, Swift, Manila**	Volumes, objets, fichiers partagés
Tout automatiser	**Heat, CLI, Terraform**	La même API REST pour tous les outils